

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники

Лабораторная работа по
Вычислительной математике №2
Вариант — метод Гаусса

Работу выполнил:

Гуменник Петр Олегович

Группа:

P3233

Санкт-Петербург,

Текст задания:

Solve system of linear algebraic equations by implementing Gauss Elimination method. Also calculate the residual errors. Format of input data: n a_{11} a_{12} ... a_{1n} b_1 a_{21} a_{22} ... a_{2n} b_2 ... a_{n1} a_{n2} ... a_{nn} b_n Output format: x_1 x_2 ... x_n r_1 r_2 ... r_n where $x_1..x_n$ - values of unknown and $r_1..r_n$ is residual errors. For systems which does not have any solutions or have unlimited number of solutions, the only following message should be printed: "The system has no roots of equations or has an infinite set of them." To do it, specify variables `isSolutionExists` and `errorMessage`.

Описание численного метода:

Метод Гаусса - это алгоритм для решения систем линейных алгебраических уравнений. Он состоит из двух этапов: прямого хода и обратного хода.

Прямой ход:

1. Выбор опорного элемента: На каждом шаге выбирается элемент с наибольшим абсолютным значением в текущем столбце. Строка, содержащая этот элемент, становится опорной.
2. Нормализация опорной строки: Опорная строка делится на опорный элемент, чтобы сделать его равным 1.
3. Исключение неизвестных: Используя опорную строку, остальные строки преобразуются так, чтобы коэффициенты при текущем неизвестном стали равны нулю.

Обратный ход:

1. Вычисление последнего неизвестного: Из последнего уравнения, где осталось только одно неизвестное, вычисляется его значение.
2. Последовательная подстановка: Найденное значение подставляется в предыдущие уравнения, и таким образом, шаг за шагом, вычисляются все остальные неизвестные.

Вычисление невязок:

После нахождения решения системы, можно вычислить невязки для каждого уравнения. Невязка - это абсолютная величина разности между левой и правой частями уравнения, когда в него подставлено найденное решение. Невязки показывают, насколько точно найденное решение удовлетворяет исходной системе уравнений.

Код программы:

```
class Solution
    isSolutionExists = True
    errorMessage = ""

    @staticmethod
    def solveByGauss(n, matrix)
        original_matrix = [[matrix[i][j] for j in range(n + 1)] for i in range(n)]

        for i in range(n)
            max_el = abs(matrix[i][i])
            max_row = i
```

```

for k in range(i + 1, n)
    if abs(matrix[k][i]) > max_el
        max_el = abs(matrix[k][i])
        max_row = k

if max_el == 0
    Solution.isSolutionExists = False
    Solution.errorMessage = "The system has no roots of equations or has an infinite set of them."
    return []

matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

for k in range(i + 1, n)
    c = -matrix[k][i] / matrix[i][i]
    for j in range(i, n + 1)
        if i == j
            matrix[k][j] = 0
        else
            matrix[k][j] += c * matrix[i][j]

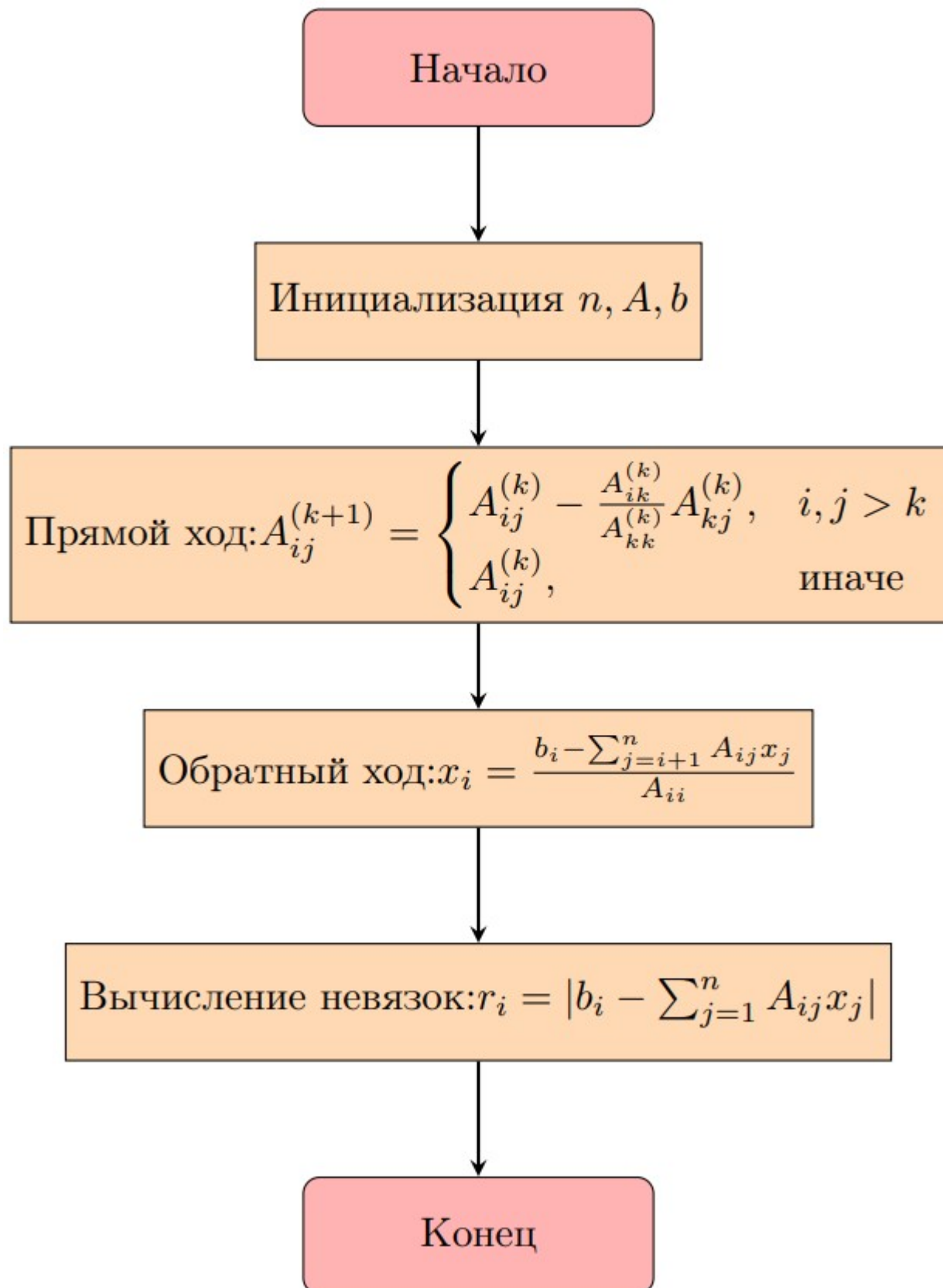
x = [0 for _ in range(n)]
for i in range(n - 1, -1, -1)
    x[i] = matrix[i][n] / matrix[i][i]
    for k in range(i - 1, -1, -1)
        matrix[k][n] -= matrix[k][i] * x[i]

residuals = []
for i in range(n)
    lhs = sum(original_matrix[i][j] * x[j] for j in range(n))
    residual = abs(lhs - original_matrix[i][n])
    residuals.append(residual)

return x + residuals

```

Блок-схема:



Примеры работы программы:

1. Простая система с уникальным решением:

Input:

3

2 1 -1 8

-3 -1 2 -11

-2 1 2 -3

Output:

2.0

3.00000000000000004

-0.9999999999999999

0.0

0.0

8.881784197001252e-16

2. Система без решения:

Input :

2

1 1 1

1 1 2

Output :

The system has no roots of equations or has an infinite set of them.

3. Система с бесконечным количеством решений:

Input :

2

1 1 1

2 2 2

Output :

The system has no roots of equations or has an infinite set of them.

4. Система с большими коэффициентами:

Input :

3

100 200 -300 1000
-200 -300 400 -2000
300 400 -500 3000

Output :

10.0
0.0
-0.0
0.0
0.0
0.0

5/ Система с матрицей большой размерности

Input:

```
start = time.time()
n = 333
matrix = [[random.randint(-100, 100) for _ in range(n + 1)] for _ in range(n)]

solution = Solution.solveByGauss(n, matrix)

print("Solution (first 10 elements) ", solution[ 10])
print("Residuals (first 10 elements) ", solution[100 110])
print("Time spent in seconds ", time.time()-start)
```

Output:

Solution (first 10 elements): [-10.392366042104024, 2.281704838782407, -16.50878953709996, -7.048004935244298, -6.225987185804675, -11.25035800570986, -18.185263853489865, 13.889084123141185, 6.6856196695711665, -5.287617068929293]

Residuals (first 10 elements): [1.451663461708824, -8.617383037096422, 8.945553061298765, -7.531588639650197, 6.028328879945382, 2.3786270693001725, -8.398596538602996, -8.961469048194747, -9.796904484645928, 17.848387643291627]

Time spent in seconds: 1.068014144897461

Вывод:

Анализ результатов:

- Примеры 1 и 4: демонстрируют успешное решение СЛАУ с единственным решением.
- Примеры 2 и 3: показывают, как метод реагирует на системы, где решения нет или оно не единственное.
- Пример 4: иллюстрирует, что метод работает и с большими коэффициентами.

- Пример 5: демонстрирует способность программы работать с большими системами уравнений. Правда точность этого метода уже не такая хорошая.

Сравнение с другими методами:

Метод Холецкого:

- Преимущества:
 - Более эффективен для СЛАУ с симметричными и положительно определенными матрицами.
 - Обладает меньшей вычислительной сложностью ($O(n^3/3)$ против $O(n^3)$ у Гаусса).
- Недостатки:
 - Не применим к несимметричным или неопределенным матрицам.

- Метод простых итераций:

- Преимущества:
 - Проще в реализации.
 - Может быть эффективен для разреженных матриц.
- Недостатки:
 - Сходимость не гарантирована.
 - Может быть очень медленным для некоторых СЛАУ.

Применимость метода:

Метод Гаусса применим к любым СЛАУ, но его эффективность и точность могут быть ограничены в следующих случаях:

- СЛАУ с большой размерностью:
 - В таких случаях метод может быть вычислительно дорогим и неустойчивым.
- Плохо обусловленные матрицы:
 - Матрицы с малым определителем могут привести к большим ошибкам в решении.

Алгоритмическая сложность:

Алгоритмическая сложность метода Гаусса составляет $O(n^3)$, где n - размерность СЛАУ.

Численная ошибка:

На численную ошибку метода Гаусса влияют:

- Ошибки округления:*

- возникают из-за ограниченной точности представления чисел в компьютере.

- Неустойчивость метода:

- может быть вызвана делением на малые числа или вычитанием близких по величине чисел.

Вычисление невязок:

Вычисление невязок позволяет оценить точность полученного решения.

Заключение:

Метод Гаусса - это простой и эффективный метод решения СЛАУ, но его применение ограничено в случае больших или плохо обусловленных систем.